Language Models

What is Language Modeling

- Language modeling (LM) forms the backbone of natural language processing (NLP), enabling machines to understand, generate, and predict human language. At its core, a language model calculates the probability of sequences of words (tokens), predicting the next token based on preceding context.
- A language model (LM) is like a smart autocomplete system: it predicts what comes next in a sentence.
- example:

  "Mother said grandma was ill, so last Sunday, Jane and I visited _____."

  A robust LM predicts "her" by analyzing contextual relationships between "grandma," "ill," and "visit"

What is Language Modeling

- A language model assigns a probability to a sequence of words.
- Say you have a sentence: “The weather is nice today”

  P("The", "weather", "is", "nice", "today") =

  P("The") × P("weather" | "The") × P("is" | "The weather") × …

- So that it can:

  Predict the next word in a sentence

  Score how "natural" a sentence is

  Generate coherent text by sampling word-by-word

What is Language Modeling

- Input:

  “The food at this restaurant is”

- A language model might output:

  “Delicious” 9.4%

  “Terrible” 3.4%

  “overpriced but good” 1.2%

- It does this by choosing the word with the highest probability at each step.

How it works ?

- **Starting Point (Prompt):** You give the model a beginning – a word, a phrase, a sentence, or even just a topic. This is like giving the musician the first few notes of a melody.
- **Predict and Extend:** The model then predicts the most probable next word (or a few probable words).
- **Sampling (The Element of Surprise):** Here's where it gets interesting. Instead of always picking the most probable word, which can lead to repetitive or generic text, we often introduce an element of randomness. This is called sampling.
- **Looping:** The selected word then becomes part of the new input, and the process repeats. The model keeps predicting and extending until it reaches a desired length, encounters a stopping condition (like a period), or you explicitly tell it to stop.

Applications

- **Creative Writing:** Generating poems, stories, scripts.
- **Content Creation:** Writing articles, blog posts, marketing copy.
- **Chatbots and Virtual Assistants:** Crafting natural-sounding responses.
- **Code Generation:** Believe it or not, even helping write software code.
- **Summarization:** Condensing long texts into shorter versions.
- **Translation:** Though more complex, language models are foundational to modern translation systems.

History of language models

1. **Rule Based Systems**

   an attempt to define the "rules" of language explicitly, through human expertise, to enable a machine to understand or generate text.

   Predefined grammar
   Couldn’t handle real-world variation

History of language models

2. **N Gram Models (statistical: pre 2010)**

Predict next words using local sequences.

Bigram: Uses one preceding word (e.g., "visited" → "her").

Trigram: Uses two preceding words.

Limitations: Cannot capture long-range dependencies; fail with rare sequences

Example:

A bigram model (N=2) only uses one previous word:

“I love” → “you” (70%), “pizza” (30%)

These models have no long term memory thats why they fail

History of language models

3. Neural Models NNLM(2010s–Present)

Next evolution: feed sentences into a neural network (usually LSTM, GRU or transformers)

Advantages:

- Captures semantic relationships (king ≈ queen)
- Handles variable-length input

History of language models

LLMs

Trained on terabytes of data with billions of parameters (e.g., BERT: 110M; GPT-3: 175B

Emergent Abilities: Exhibit unexpected skills like code generation and few-shot learning

Transformers like GPT are autoregressive:

Predict the next word using only the previous ones

Repeats this until it reaches a max length or special stop token. No peeking ahead!

Gpt2 text gen

```
from transformers import GPT2Tokenizer, GPT2LMHeadModel
import torch

# Load pre-trained model and tokenizer
model = GPT2LMHeadModel.from_pretrained("gpt2")
tokenizer = GPT2Tokenizer.from_pretrained("gpt2")

# Prepare input prompt
prompt = "The future of education is"
input_ids = tokenizer.encode(prompt, return_tensors="pt")

# Generate text
output_ids = model.generate(input_ids, max_length=50)
print(tokenizer.decode(output_ids[0], skip_special_tokens=True))
```

Decoding Stratergies

- Greedy Search
- Beam Search
- Top-K Sampling
- Top-P (Nucleus) Sampling

1. Greedy Search

- Mechanism: Always selects the token with the highest probability at each step.
- Example:

  Input: "The sky is" → Probabilities: blue (0.7), gray (0.2), cloudy (0.1) → Output: "blue"

- Fast and Deterministic (same output every time)
- Cons:

  Often gets stuck in repetitive or generic outputs

  May miss better options later in the sentence

```
model.generate(input_ids, max_length=50, do_sample=False)
```

2. Beam Search

- Instead of picking just the best word, it keeps track of top N sequences at each step.
- This is like solving a maze and keeping multiple paths alive until the end.
- Pros:

  Higher quality output

  Balances exploration and exploitation

- Cons:

  Slower than greedy

  Still deterministic

  May result in similar or generic outputs for all beams

```
model.generate(input_ids, max_length=50, num_beams=5, early_stopping=True)
```

3. Top k-sampling

- At each step, randomly choose the next word from the top K words only.
- Pros:

  More creative outputs

  Controls randomness with top_k size
- Cons:

  Too low k = limited diversity

  Too high k = possible nonsense
- Top-5 means you ignore everything outside the best 5 predictions

```
model.generate(input_ids, do_sample=True, top_k=50, max_length=50)
```

4. Top-P Sampling (Nucleus Sampling)

- Choose next word randomly from the smallest set of words whose cumulative probability ≥ p (like 90%).
- Pros:

  Dynamic: adapts the number of candidates based on confidence

  Often produces human-like, diverse outputs

- Top-P is great for storytelling, chatbots, poetry
- Cons:

  Less predictable

  Sensitive to temperature and p

```
model.generate(input_ids, do_sample=True, top_p=0.9, max_length=50)
```

Temperature

- Another parameter you can add to sampling methods
- Sharpen (T < 1) or flatten (T > 1) the probability distribution before sampling => Control randomness.
- T = 0.3 → "The sky is blue" (deterministic)
- T = 1.0 → "The sky is azure" (balanced)
- T = 2.0 → "The sky is singing" (unpredictable)

```
model.generate(input_ids, top_p=0.9, do_sample=True, temperature=0.7)
```

comparison

Strategy	Random?	Creative	Use Case
Greedy	❌	❌	Fast, baseline testing
Beam	❌	❌	High-quality formal output
Top-K	✅	✅	Casual, creative writing
Top-P	✅	✅✅	Conversational AI, poems
Combo	✅✅	✅✅	Best of both worlds

Text Evaluation metrics

Why text evaluation metrics are a bit different ?

- Unlike classification (where you just check true vs false), text generation has:

  Multiple valid outputs for the same input

  Subjective quality (fluency, creativity, relevance)

- For example:

  Prompt: “The capital of France is”

  Output A: “Paris”

  Output B: “Paris, the largest city in France”

  Output C: “France’s capital is Paris.”

- Above all are corect, but token-wise comparison may say they’re different.
- So we need smart metrics.

BLEU (Bilingual Evaluation Understudy)

- Imagine you have trained a young apprentice translator. You give them a foreign sentence, and they give you their translation. Now, how do you judge if their translation is good? You'd likely compare it to a high-quality translation done by a seasoned expert, wouldn't you? You'd look for matching words, phrases, and the overall fluency.
- BLEU (Bilingual Evaluation Understudy) score is precisely that: a metric designed to automatically evaluate the quality of machine-translated text by comparing it to one or more human-created reference translations.

BLEU (Bilingual Evaluation Understudy)

- While it was initially conceived for machine translation, its utility quickly expanded to other text generation tasks, including summarization, image captioning, and even general text generation, where you have a "reference" text that the generated output should ideally resemble.
- Example =>

  Your student writes an essay. You, the Teacher, have a "model answer" or a rubric. You compare the student's essay word by word, phrase by phrase, to your model. The more matches (and the longer the matching phrases), the better the essay is considered. BLEU does something very similar, but for language.

- BLEU is primarily a precision-based metric. This means it focuses on how many words and word sequences (n-grams) in the machine-generated text also appear in the reference translation. It doesn't penalize the machine for missing words that are in the reference but not in its output (that would be recall, which BLEU doesn't directly measure in its main component).

BLEU (Bilingual Evaluation Understudy)

- **N-gram Precision (P_n):**

  This is the core of BLEU. An "n-gram" is a contiguous sequence of 'n' words.

  1-gram (unigram) precision: How many individual words in the machine translation appear in the reference?

  2-gram (bigram) precision: How many two-word sequences appear?

  And so on

- To prevent a machine translation that just repeats a single relevant word many times from getting a high score, BLEU uses a "clipped count."

  If a word appears 3 times in the machine translation but only 1 time in the reference, its count is "clipped" to 1. You only get credit for matches up to the maximum number of times that n-gram appears in any of the references. This prevents gaming the system.

ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

- While BLEU focuses on precision to evaluate machine translation, ROUGE (Recall-Oriented Understudy for Gisting Evaluation) shifts its focus to recall, making it particularly suitable for tasks like summarization where the goal is to capture as much important information from the source as possible.
- ROUGE is a set of metrics designed to automatically evaluate the quality of a machine-generated summary (or other text) by measuring the overlap between it and one or more human-created reference summaries, with a primary focus on recall.
- Recall (R): What percentage of the words/n-grams in the reference summary are also present in the candidate summary? This is the primary focus of ROUGE.

METEOR (Metric for Evaluation of Translation with Explicit ORdering)

- If BLEU is focused on precision and ROUGE on recall, METEOR aims for a more balanced and linguistically informed evaluation.
- METEOR was developed to address this limitation by incorporating linguistic knowledge, such as stemming and synonymy, and by explicitly considering word order, providing a better correlation with human judgments of translation quality.
- Uses WordNet for similarity
- Better correlation with human judgment
- Analogy : The Smart Librarian: A librarian doesn't just match books by their exact title. They understand synonyms ("book" vs. "tome"), stemming ("run" vs. "running"), and even paraphrases. METEOR is like that smart librarian, looking for matches that go beyond literal word forms.

Perplexity

- Measures how well a model predicts a sequence
- Lower is better
- Not for comparing outputs, but for model evaluation

Which metrics to use when

Metric	Good For	Handles Synonyms?	Short Texts?
BLEU	Translation	❌	❌
ROUGE	Summarization	❌	✅
METEOR	Paraphrasing	✅	✅
Perplexity	Model Performance	❌	✅